

Scrubless — Architecture

Semantic video search · *Stop scrubbing. Start searching.*

Ernest Opara

2026-05-22

Contents

1 Overview	2
1.1 Design principles	2
2 Technology stack	3
3 System architecture	4
4 Indexing pipeline	5
5 Search pipeline	6
6 Data model	7
6.1 Relational (users table — SQLAlchemy)	7
6.2 Vector (ChromaDB segments collection)	7
6.3 In-memory (VIDEOS dict)	7
7 API reference	8
8 Accounts, tiers & monetization	9
9 Configuration (environment variables)	10
10 Deployment	11
10.1 Project layout	11
11 Known limitations (V1)	12
12 Roadmap — V2 (weekend of 2026-05-23)	13
13 Change log	14

1 Overview

Scrubless is a semantic video-search product. The entire user story is one sentence:

Upload a video → index it → search it with natural language → click a result and the player jumps to that exact moment.

Under the hood, every uploaded video is broken into still frames and an audio transcript. Each frame is turned into a CLIP embedding (a vector that captures what the frame *looks like*) and stored in a vector database alongside the transcript text for that moment. A search query is embedded the same way, and the database returns the frames whose meaning is closest to the query.

The guiding design constraint (from `CLAUDE.md`) is **radical simplicity**: one language (Python), one backend file (`app.py`), one frontend file (`index.html`), no Docker-for-dev, no microservices, no build step.

1.1 Design principles

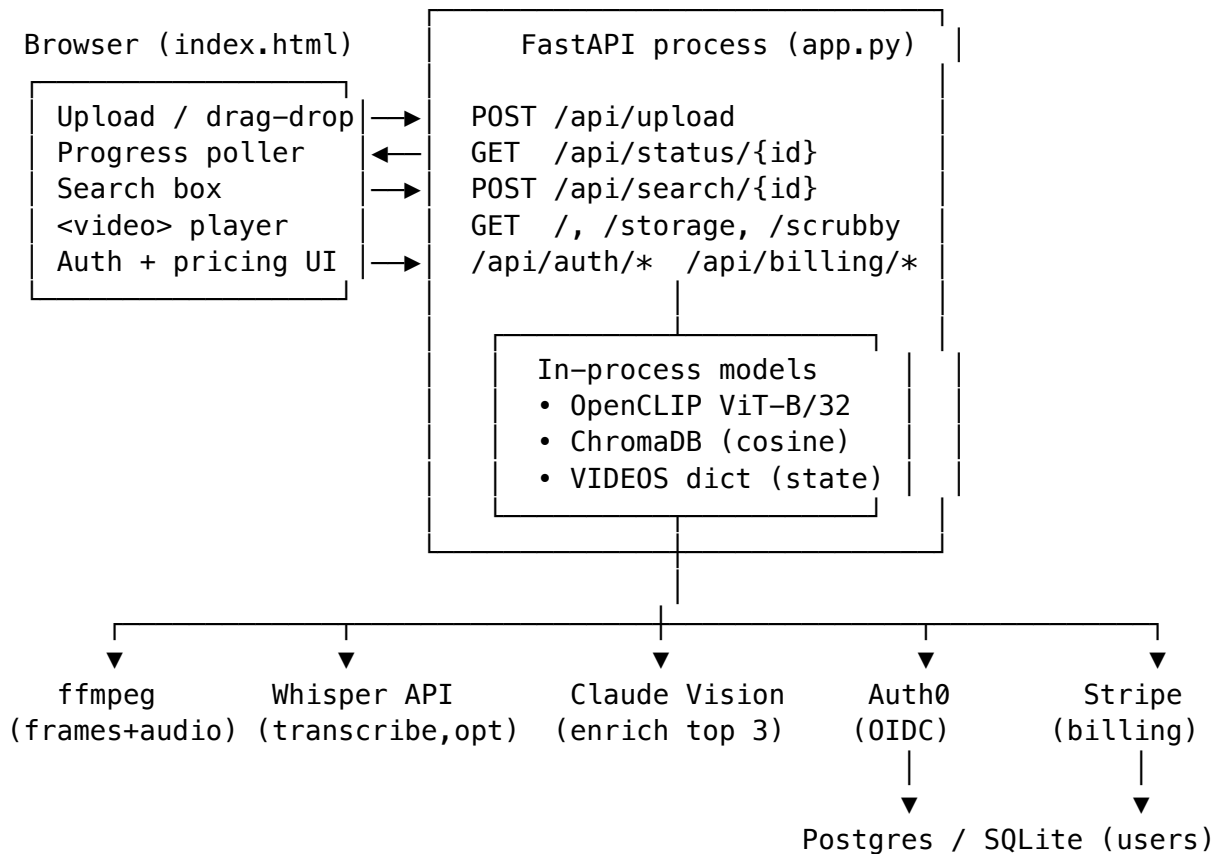
- **One file per layer.** All backend logic lives in `app.py`; all UI lives in `index.html`. No framework scaffolding, no orchestration.
- **In-process everything.** The CLIP model and the vector database both run inside the FastAPI process — no sidecars, no network hops.
- **Graceful degradation.** Every external dependency (Whisper, Claude, Auth0, Stripe, Postgres) is optional. Missing a key disables that feature instead of crashing the app.
- **Cheap to run.** Designed to fit on a single small VPS / one Railway service.

2 Technology stack

Concern	Choice
Web framework	FastAPI (single file: <code>app.py</code>)
ASGI server	uvicorn
Video processing	ffmpeg via subprocess (frames + audio)
Visual embeddings	OpenCLIP ViT-B/32 (laion2b_s34b_b79k)
Vector search	ChromaDB (in-process <code>PersistentClient</code> , cosine)
Transcription	OpenAI Whisper API (<code>whisper-1</code>) — optional
Result enrichment	Anthropic Claude Vision (<code>claude-sonnet-4-6</code>)
Frontend	One <code>index.html</code> — vanilla JS + CSS, no build step
Auth	Auth0 (OIDC) via Authlib + signed session cookies
Accounts DB	SQLAlchemy 2.0 — SQLite (dev) / Postgres (prod)
Billing	Stripe Checkout + Customer Portal + webhooks
Deployment	Docker image on Railway; custom domain via GoDaddy

Explicitly *not* used: Go, Docker Compose for dev, Terraform/AWS/ECS/S3, React build pipeline, face recognition, knowledge graphs, YouTube video downloads, clip export (these are deferred or rejected per `CLAUDE.md`).

3 System architecture



The FastAPI process is the whole backend. The only stateful stores are:

1. **ChromaDB** on disk (storage/chroma) — frame embeddings + metadata.
2. **Relational DB** (users table) — accounts, tiers, Stripe IDs.
3. **VIDEOS** — an in-memory dict of per-video processing status (lost on restart, which is acceptable for V1).

Everything else (ffmpeg, Whisper, Claude, Auth0, Stripe) is an external call.

4 Indexing pipeline

When a video is uploaded, `process_video()` runs in a background thread:

1. **Frame extraction** — `extract_frames()` runs `ffmpeg` with `fps=1/5` to grab one JPEG every **5 seconds** (`FRAME_INTERVAL`) into `storage/{id}/frames/`.
2. **Audio extraction** — `extract_audio()` produces a 16 kHz mono MP3 (48k bitrate) small enough for the Whisper API. Wrapped in `try/except`: videos with no audio track index visually and continue.
3. **Transcription** — `transcribe()` calls `Whisper(verbose_json)` and returns `[{start, end, text}]` segments. Skipped silently if `OPENAI_API_KEY` is absent.
4. **Embedding** — each frame is CLIP-encoded (`embed_image`), L2-normalized, and paired with the transcript text overlapping its 5-second window (`transcript_for_window`).
5. **Storage** — all frames are written to ChromaDB in one `segments.add()`: `id = "{video_id}-{i}"`, the embedding vector, metadata `{video_id, timestamp, frame_path, transcript_segment}`, and the transcript snippet as the document.
6. **Status** — progress is updated per frame; status flips to `indexed` when done, or error with a message on failure.

5 Search pipeline

POST /api/search/{video_id} does the following:

1. Reject if the video isn't indexed yet (409) or the query is empty (400).
2. **Embed the query** with `embed_text()` (same CLIP space as the frames).
3. **Vector query** ChromaDB for the top **10** results, filtered to the one `video_id`, ranked by cosine distance.
4. Convert distance to a similarity **score** ($1.0 - \text{dist}$) and attach the frame URL, timestamp, and transcript snippet.
5. **Enrich** the top **3** results (ENRICH_TOP_N) with a 1–2 sentence Claude Vision description of the frame (best-effort; skipped without an Anthropic key).
6. Return the list; the frontend renders thumbnails the user can click to seek the <video> player.

6 Data model

6.1 Relational (users table — SQLAlchemy)

Column	Type	Notes
id	int PK	local user id
auth0_sub	str	Auth0 subject, unique + indexed
email	str	from the OIDC profile
tier	str	free / pro / studio
stripe_customer_id	str	set on first checkout
stripe_subscription_id	str	tracked via webhooks
created_at	float	unix timestamp

DATABASE_URL selects the backend: SQLite file in dev, Postgres in prod (postgres:// is normalized to postgresql://).

6.2 Vector (ChromaDB segments collection)

Per frame: an embedding vector + metadata {video_id, timestamp, frame_path, transcript_segment} and the transcript snippet as the searchable document. Distance space: **cosine** (hnsw:space).

6.3 In-memory (VIDEOS dict)

Per video: {status, progress, total_segments, error, source, title, created, owner}. Ephemeral — rebuilt only for the sample video on restart.

7 API reference

Method	Path	Purpose
GET	/	Serve index.html
POST	/api/upload	Upload a video, start indexing (cap-gated)
GET	/api/status/{id}	Poll indexing status / progress
POST	/api/search/{id}	Natural-language search of one video
DELETE	/api/videos/{id}	Delete a video (sample is protected)
GET	/api/auth/login	Redirect to Auth0 Universal Login
GET	/api/auth/callback	OIDC callback → create/find user, set session
GET	/api/auth/logout	Clear session + Auth0 logout
GET	/api/auth/me	Current user, upload cap, feature flags
POST	/api/billing/checkout	Create a Stripe Checkout session
POST	/api/billing/portal	Open the Stripe Customer Portal
POST	/api/billing/webhook	Stripe webhook → update tier
—	/storage/*, /scrubby/*	Static mounts (frames/video, mascot)

8 Accounts, tiers & monetization

Access is gated by upload size. Anonymous visitors are treated as the free tier.

Tier	Upload cap	Price	Account required
Free	500 MB	\$0	No (anonymous)
Pro	2 GB	\$10 / mo	Yes
Studio	10 GB	\$30 / mo	Yes

- **Caps** are enforced twice in `/api/upload`: a fast Content-Length reject, then a streaming backstop that deletes the partial file if the body overruns.
- **Free-tier hygiene**: anonymous uploads auto-expire after **24h** (`expiry_sweep`, every 30 min). The sample video and account-owned videos are never expired.
- **Login** is Auth0 OIDC via Authlib; the session is a signed cookie (`SessionMiddleware`). New subjects create a User row on first callback.
- **Billing** is Stripe hosted Checkout (subscription mode). Tier changes are driven by `customer.subscription.*` webhooks (signature-verified), keyed by Stripe customer id. The Customer Portal handles upgrades/cancellation.

9 Configuration (environment variables)

Variable	Enables / controls
OPENAI_API_KEY	Whisper transcription (else visual-only)
ANTHROPIC_API_KEY	Claude Vision result enrichment
SESSION_SECRET / AUTH0_SECRET	Signed session cookie key
APP_BASE_URL	Public base URL for OAuth/Stripe redirects
AUTH0_DOMAIN	Auth0 tenant (scheme/slash tolerated)
AUTH0_CLIENT_ID	Auth0 application client id
AUTH0_CLIENT_SECRET	Auth0 application client secret
DATABASE_URL	SQLite (dev) or Postgres (prod) connection
STRIPE_SECRET_KEY	Enables billing
STRIPE_WEBHOOK_SECRET	Verifies webhook signatures
STRIPE_PRICE_PRO	Stripe price id for the Pro plan
STRIPE_PRICE_STUDIO	Stripe price id for the Studio plan

Feature flags derived at startup: AUTH0_ENABLED (all three Auth0 vars present), STRIPE_ENABLED (STRIPE_SECRET_KEY present). Missing keys downgrade the feature, never crash the app.

10 Deployment

- **Container:** Dockerfile on python:3.11-slim, installs ffmpeg, pip-installs requirements.txt, pre-bakes CLIP weights into the image, exposes 8080, runs uvicorn.
- **Host:** Railway (one service) defined by railway.toml. .dockerignore keeps storage/, .env, .venv/, and legacy/ out of the image.
- **Domain:** getscrubless.com via GoDaddy DNS — www CNAME → Railway, apex 301-forwarded, _railway-verify.www TXT for verification. TLS issued by Railway.
- **Persistence caveat:** Railway's filesystem is ephemeral, so the sample video is re-indexed on every boot (ingest_sample) and Postgres (DATABASE_URL = \${Postgres.DATABASE_URL}) is required for durable accounts.

10.1 Project layout

```
clipfind/
├── app.py                # entire backend
├── index.html            # entire frontend
├── requirements.txt
├── Dockerfile, railway.toml, .dockerignore
├── assets/sample.mp4 # auto-indexed demo clip (Big Buck Bunny)
├── scrubby/            # mascot SVGs + favicon
├── docs/ARCHITECTURE.md # this document
├── storage/            # runtime: uploads, frames, chroma (gitignored)
└── legacy/            # archived Go + React + sidecar stack
```

11 Known limitations (V1)

- **Restart loses upload state.** VIDEOS is in-memory; only the sample is rebuilt. Embeddings persist in ChromaDB, but the per-video status map does not.
- **5-second frame granularity.** Moments shorter than the sampling interval can be missed.
- **Single-process / single-node.** No horizontal scaling; CLIP runs on CPU.
- **Local file storage.** Not backed by object storage — ephemeral on Railway.
- **No clip export, no YouTube ingest, no face recognition** (deferred by spec).

12 Roadmap — V2 (weekend of 2026-05-23)

Theme: from demo □ product. V1 is shipped, deployed, and monetized; V2 makes it durable and sticky.

Foundation (committed):

1. **Durable storage + state.** Move uploads/frames/Chroma onto a Railway persistent volume and persist per-video state to Postgres, so indexed videos survive restarts (today only the sample is rebuilt; VIDEO5 is in-memory).
2. **Per-user library.** Logged-in users see and re-search their own indexed videos (reopen / re-search / delete). Anonymous uploads stay ephemeral (24h).

Marquee feature (chosen): **Library Mode — search across a whole folder.**

Turn Scrubless from a single-clip tool into a search engine for a video *library*: one natural-language query returns the best moments across **every** video in a directory, each result identifying its source file + timestamp.

- **Cross-video search.** Frames are already tagged with `video_id` in ChromaDB, so searching a collection is mostly relaxing the per-video where filter and carrying the source video into each result. The expensive part (per-video indexing) already exists.
- **Collections.** Add a `collection_id` grouping so a query scopes to “this folder” / “all my videos.”
- **Batch ingestion** — two modes feeding the same pipeline:
 - *Local directory scan* (self-hosted): point at a path (e.g. `~/Videos/footage`), enumerate video files, index in place — nothing uploaded. Private, fast, handles large folders; fits “run locally.”
 - *Hosted folder upload*: drag a whole folder into the site; index all, bounded by tier caps/storage.
- **Constraint:** indexing is CPU-bound (CLIP on CPU); a large folder takes time on first index — durable state + a progress UI (the committed foundation) make that acceptable.

Both ingestion modes are committed for V2, **local directory scan first**, then hosted folder-upload reusing the same cross-video engine + collections.

Deferred (former marquee options + later work): clip export + share links, YouTube ingest, search-quality (hybrid ranking), React rewrite, face recognition (legal review), native mobile, GPU inference.

13 Change log

Per the standing instruction, this document is updated on **every prompt**. Each entry records the prompt that triggered the update and what changed.

Date	Prompt (summary)	Change to this doc
2026-05-22	"Create a PDF on the project architecture; update it every prompt."	Initial architecture document created and rendered to PDF.
2026-05-22	"Can I fix the Auth0 login showing 'dev-4qodgus8'?"	No architecture change — Auth0 <i>tenant friendly name</i> dashboard setting (cosmetic Universal Login branding), not a code/config change in the app.
2026-05-22	"Welcome / Log in to Scrubless..." (confirming the fix).	No architecture change — confirmed Auth0 friendly-name now reads "Scrubless"; remaining redundancy is template wording, optionally overridable via Universal Login Custom Text.
2026-05-22	"Alerts: connections using Auth0 development keys..."	No architecture change — Auth0 social connection (Google) on shared dev keys; fix is supplying your own Google OAuth client id/secret in the Auth0 Google connection (dashboard only), or disabling the Google button. Not for production on dev keys.
2026-05-22	"okay it is gone" (dev-keys alert cleared).	No architecture change — Auth0 setup now complete (login + branding + own Google keys). Remaining prod-config items: set DATABASE_URL=\${{Postgres.DATABASE_URL}} (durable accounts) and switch Stripe to Test mode (key/price IDs/webhook secret) so the 4242 card works.
2026-05-22	"what is v2 for this weekend"	Added Roadmap — V2 section: committed foundation (durable storage/state + per-user library) and a pending marquee choice (clip export / YouTube ingest / search quality).
2026-05-22	"what if we can use scrubless on a directory full of videos"	Reframed V2 marquee to Library Mode : cross-video search across a folder/collection + batch ingestion, reusing the existing per-video index + video_id metadata. Local directory scan = primary target; hosted folder-upload reuses the same engine. Former options (clip export / YouTube / search-quality) deferred.

Date	Prompt (summary)	Change to this doc
2026-05-22	"i want to do both but local first. also commit v1 totally."	Locked V2 ingestion: both modes, local directory scan first then hosted folder-upload. Committed docs/ (architecture doc + PDF + render script) and tagged the V1 milestone v1.0.